Relazione del Progetto Wheel Of Fortune

mercoledì 8 luglio 2026 17:54

Fouad Rabichi
Matricola: 20059586

Enoch Panzu Mfumu
Matricola: 20060690

Funzionalità implementate

Funzionalità	Discussa nella sezione
1.1 Versione classica del gioco	2.1-2.2
2.1 Supporto modalità di gioco differenti --mode	2.3
2.2 Caricamento librerie frasi esterne --lib	2.4
2.3 Ripetibilità del codice tramite --seed	2.5
2.4 Ignorare maiuscole minuscole nell'input	2.6
2.5 Validazione input dei giocatori	2.6
2.6 Eccezioni specifiche e concatenamento	2.7
2.8 Arrotondamento del denaro	2.8
2.7 Diagramma delle classi UML	3

2.1 Scelte progettuali e architetturali

Il progetto è stato suddiviso in vari package per favorire la modularità del codice e la **separazione delle responsabilità**. In questo modo, ogni componente del sistema ha un ruolo ben definito e isolato.

Package	Utilizzo
ruotadellafortuna.main	Serve per l'avvio del gioco
ruotadellafortuna.coredelgioco	Serve per la logica di turni e round
ruotadellafortuna.modello	Serve per gli oggetti del gioco come Ruota, Spicchi e Frase, etc..
ruotadellafortuna.util	Serve per le classi di utilià personalizzate come Input e Randomizzatore

La scelta di implementare le classi di supporto **Input** e **Randomizzatore** all'interno di un package dedicato è stata presa per rispettare il concetto di **riutilizzo del codice**.
Invece di riutilizzare e duplicare le librerie Java standard e i controlli try-catch all'interno di ogni classe abbiamo deciso di centralizzare il tutto.
La classe **Input** incapsula e gestisce la logica di validazione e di eccezioni garantendo la restituzione corretta al chiamante del dato di input richiesto. Allo stesso modo **Randomizzatore** garantisce un'unica istanza condivisa tra tutte le classi per la generazione di numeri permettendo anche in maniera semplice di usa il parametro --seed per la ripetibilità della partita.
Il package modello è stato usato per agire come contenitore esclusivo delle entità fisiche del dominio di gioco, fornendo le fondamenta su cui il core del gioco si appoggia per organizzare le partite.

2.2 Modellazione degli oggetti principali(Versione Classica)

Per controllare il tipo di spicchio uscito è stato usato l'**enum tipoSpicchio**, questo garantisce la flessibilità e la sicurezza che s'intraprenda la giusta condizione in base al tipo uscito senza dover costantemente fare il casting per testare la condizione. I valori possibili sono fissi:

tipoSpicchio	Funzione
SOLDI	È uno spicchio intero che quindi significa che si è fermato sul denaro
PASSA	È lo spicchio che fa cedere il turno senza giocare al giocatore
BANCAROTTA	È lo spicchio che elimina tutto il guadagno di round del giocatore(Non tocca i soldi accumulati).
JOLLY	È lo spicchio che ti permette di evitare lo spicchio PASSA (si può possedere una volta a round)
JOLLYPLUS	È lo spicchio che ti permette di evitare oltre a PASSA anche lo spicchio BANCAROTTA (lo può possedere un solo giocatore a round una volta sola)

Abbiamo seguito il consiglio di utilizzare la classe personalizzata **UnioneStringa** per incapsulare insieme il tema e la frase estratta, questa tuttavia non si occupa di oscurare la frase, per quello c'è **Frase** che usa un array di caratteri per prendere la frase estratta e sostituire solo i caratteri con _ evitando di farlo per gli spazi " ".
La classe Giocatore è quella che si occupa esclusivamente della struttura del giocatore, quindi il suo nome, il suo numero in game e soprattutto il denaro totale accumulato durante il round e durante la partita. È la struttura fondamentale su cui si appoggia la logica di chi gioca nel turno e quanti soldi possiede. La progressione dei turni all'interno della classe Round usa il modulo (indiceTurno + 1) % giocatori.length per ciclare in maniera sicura all'interno dell'array giocatori. È stato anche deciso di implementare un metodo di sicurezza aggiuntivo che verifica e avvisa i giocatori nel caso siano rimaste solo vocali da scoprire nella frase.

2.3 Supporto alle varianti tramite Ereditarietà e Polimorfismo

Per soddisfare il requisito aggiuntivo della modalità estesa (**--mode extended**) evitanto la duplicazione del codice abbiamo optato per l'uso dell'ereditarietà. È stata creata la classe **RuotaAvanzata** che estende RuotaClassica. Sovrascrivendo unicamente il costruttore, è stato possibile ampliare l'array degli spicchi da 24 a 26 elementi per includere i due Jolly.

Nella classe **Gioco**, la scelta della ruota avviene in runtime sfruttando il **Dynamic Binding**.
La variabile **ruotaFortuna** è dichiarata col tipo della superclasse ma può comunque ospitare l'istanza della sottoclasse **RuotaAvanzata**.

2.4 Gestione dinamica delle Frasi e Java Collections

Per gestire l'aumento della difficoltà delle frasi nei vari round ed evitare i limiti di ridimensionamento degli array classici, abbiamo sfruttato la **Java Collections**.
Nella classe **TutteLeFrasi** l'achivio è modellato come una lista di liste quindi come (**list<list<unioneStringa>>**). In questo modo il sistema è flessibile al punto di poter permettere di caricare librerie esterne (**--lib**) di dimensioni variabili.
Il parsing del file di testo avviene tramite le classi **BufferedReader** e **FileReader**, dividendo le stringhe con il delimitatore #.
L'estrazione casuale garantisce l'assenza di duplicati rimuovendo la frase dalla collezione una volta estratta.

2.5 Ordine dei giocatori e ripetibilità del gioco

L'ordine inziale è casualmente generato grazie al metodo **Collections.shuffle().**
Per soddisfare il requisito di ripetibilità (**--seed**), l'intera generazione di numeri causali è delegata a **Randomizzatore**. Questa classe gestisce un'unica istanza statica di **java.util.Random**. Passando tale istanza al metodo shuffle e alle estrazioni casuali della ruota, si garantisce che a parità di seed inserito e di scelte effettuate dai giocatori, l'evoluzione della partita sia identica e quindi deterministica.

2.6 Robustezza e validazione dell'input

Il programma è stato anche progettato per essere il più possibile resistente agli errori. La classe Input centralizza l'acquisizione dei dati da tastiera avvolgendo i metodi della classe Scanner all'interno di costrutti while(true) e blocchi try-catch. In questo modo, l'inserimento di stringhe al posto di interi viene gestito senza far terminare bruscamente il programma.
Inoltre per rispettare il requisito di insensibilità a maiuscole/minuscole, ogni input testuale viene automaticamente convertito in maiuscolo tramite **toUpperCase()**.

2.7 Eccezioni personalizzate ed Exception Chaining

È stata definita l'eccezione personalizzata di tipo checked **EccezioniDelGioco** che naturalmente estende **Exception**. Tale eccezione viene sollevata al verificarsi di anomalie legate alla logica come ad esempio: l'esaurimento delle frasi disponibili o file non trovati o malformati.
Per non perdere l'eccezione originale, la classe implementa un costruttore che accetta come parametro la causa scatentante **Throwable excpetion**, supportando l'**Exception chaining.**

2.8 Calcoli monetari

Poiché i tipi primitivi double e float sono imprecisi date le loro approssimazioni, sono inadeguati a calcoli monetari, a tal proposito è stato utilizzato il **BigDecimal**.
Per soddisfare il requisito di arrotondamento, è stato creato il metodo di supporto **arrotondaSoldi()** che applica **RoundingMode.HALF_UP** in modo da arrotondare in base alla seconda cifra decimale per ogni addizione e sottrazione.

3 Diagramma UML

